

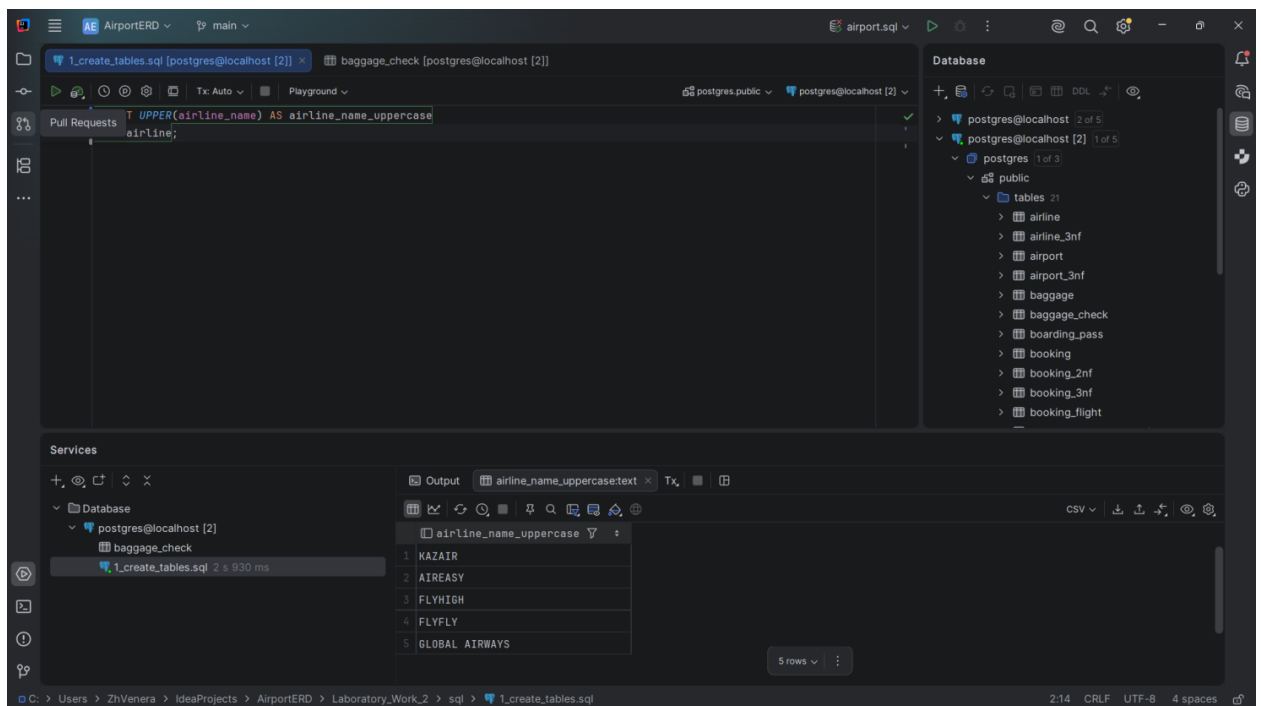
MUKHAMETKALI MEREY

Laboratory work 4

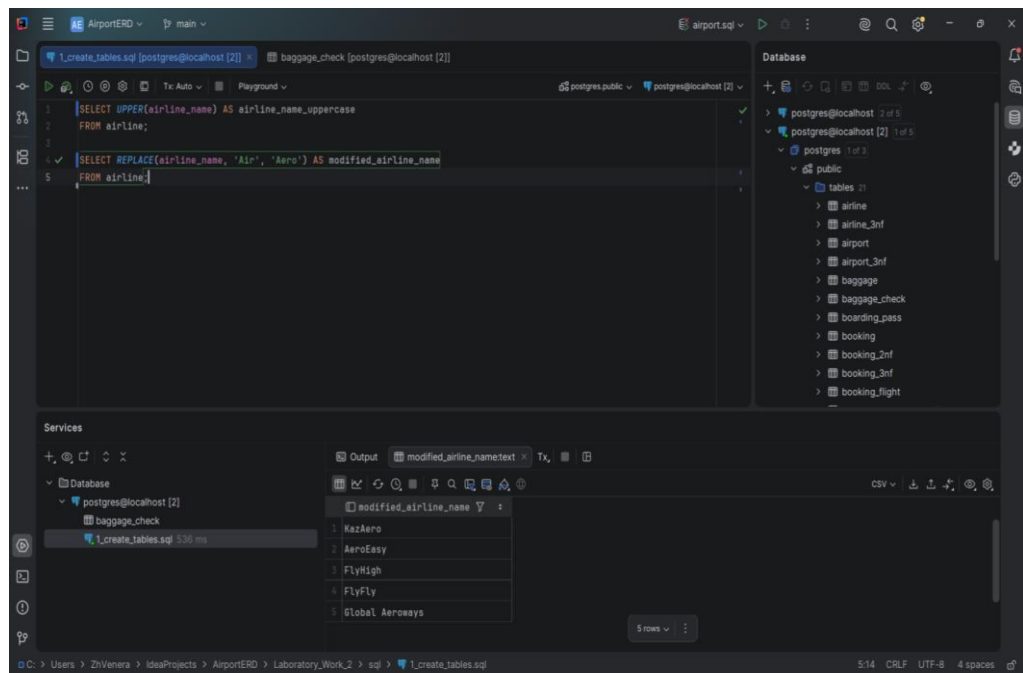
We continue to work with the database from the previous laboratory works.

Tasks:

1. Retrieve all airline names in uppercase. SELECT UPPER(airline_name) AS airline_name_uppercase FROM airline;



2. Replace any occurrence of the word "Air" in airline names with "Aero". SELECT REPLACE(airline_name, 'Air', 'Aero') AS modified_airline_name FROM airline;

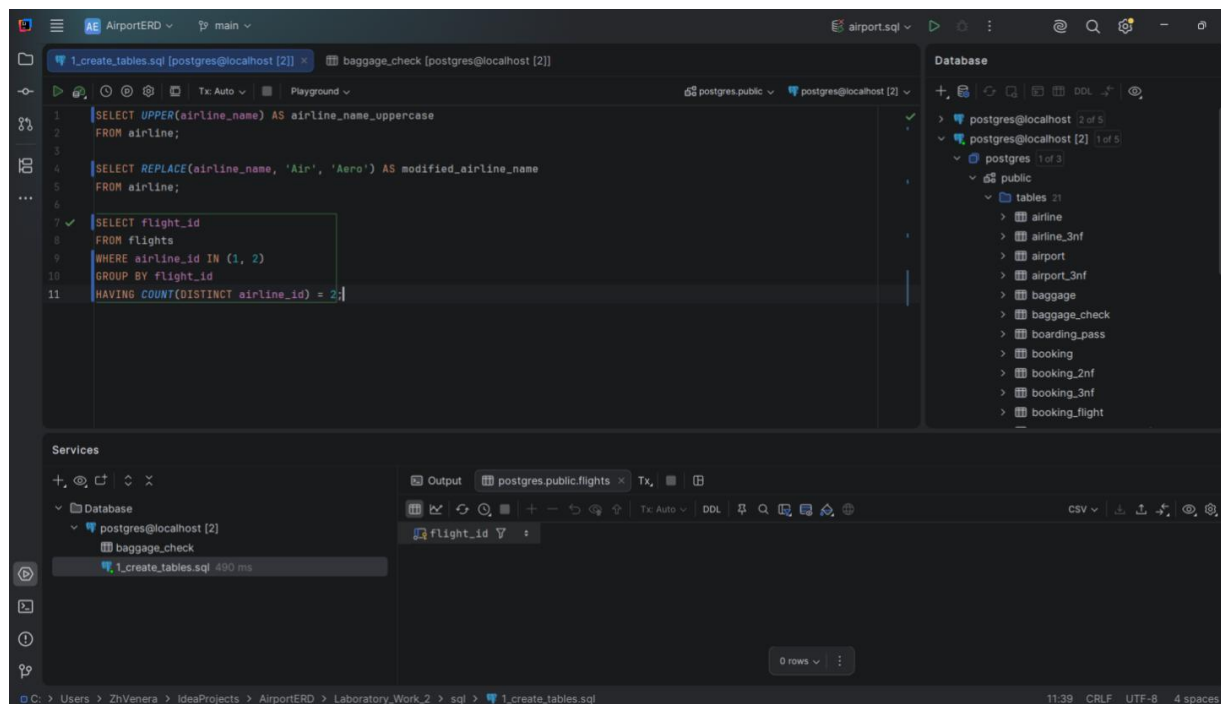


3. Find all flight numbers that coordinates with both airline 1 and airline 2.

```

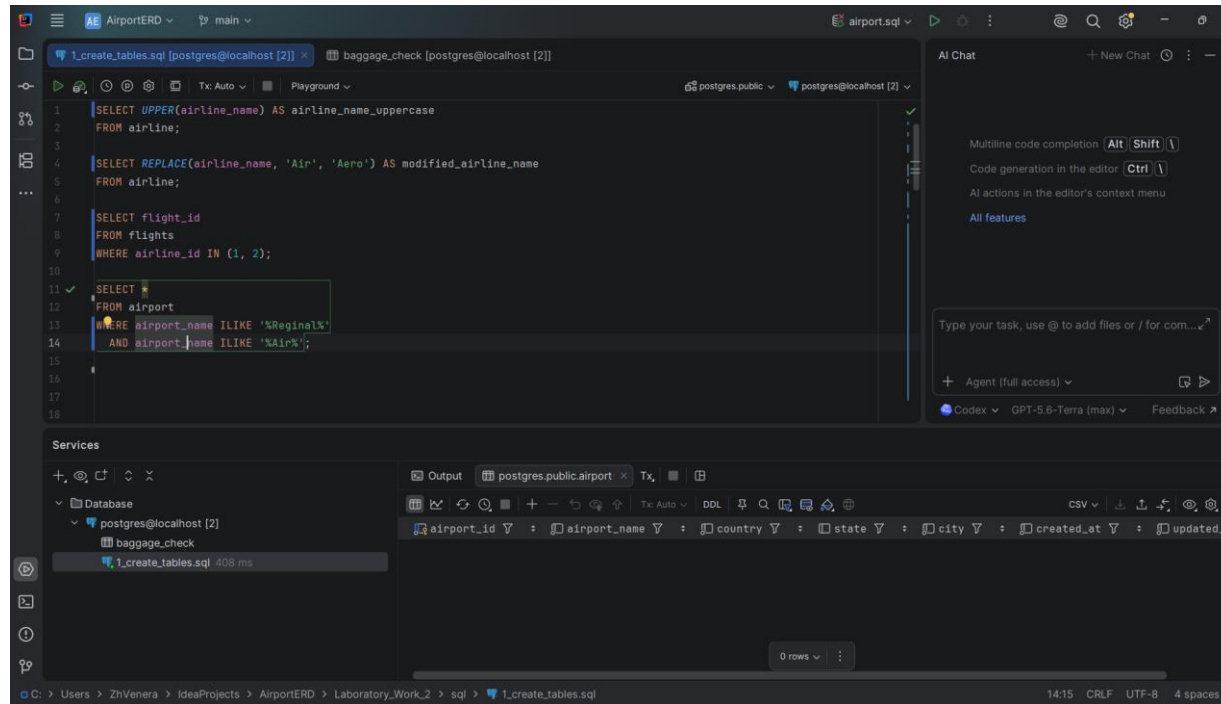
SELECT flight_id
FROM flights
WHERE airline_id IN (1, 2)
GROUP BY flight_id
HAVING COUNT(DISTINCT airline_id) = 2;

```



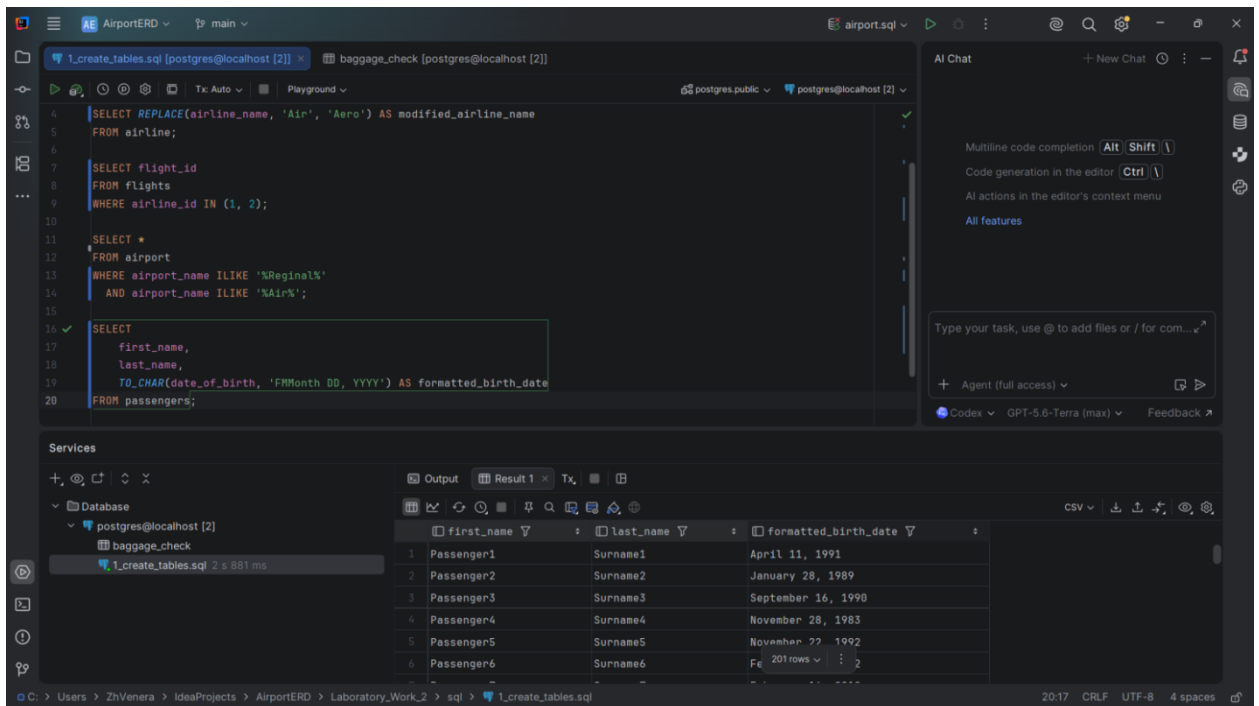
4. Retrieve airports that contain the word "Reginal" and "Air" in their names.

```
SELECT *  
FROM airport  
WHERE airport_name ILIKE '%Reginal%'  
AND airport_name ILIKE '%Air%';
```



5. Retrieve passenger names and format their birth dates as 'Month DD, YYYY'.

```
SELECT  
    first_name,  
    last_name,  
    TO_CHAR(date_of_birth, 'FMMonth DD, YYYY') AS formatted_birth_date  
FROM passengers;
```

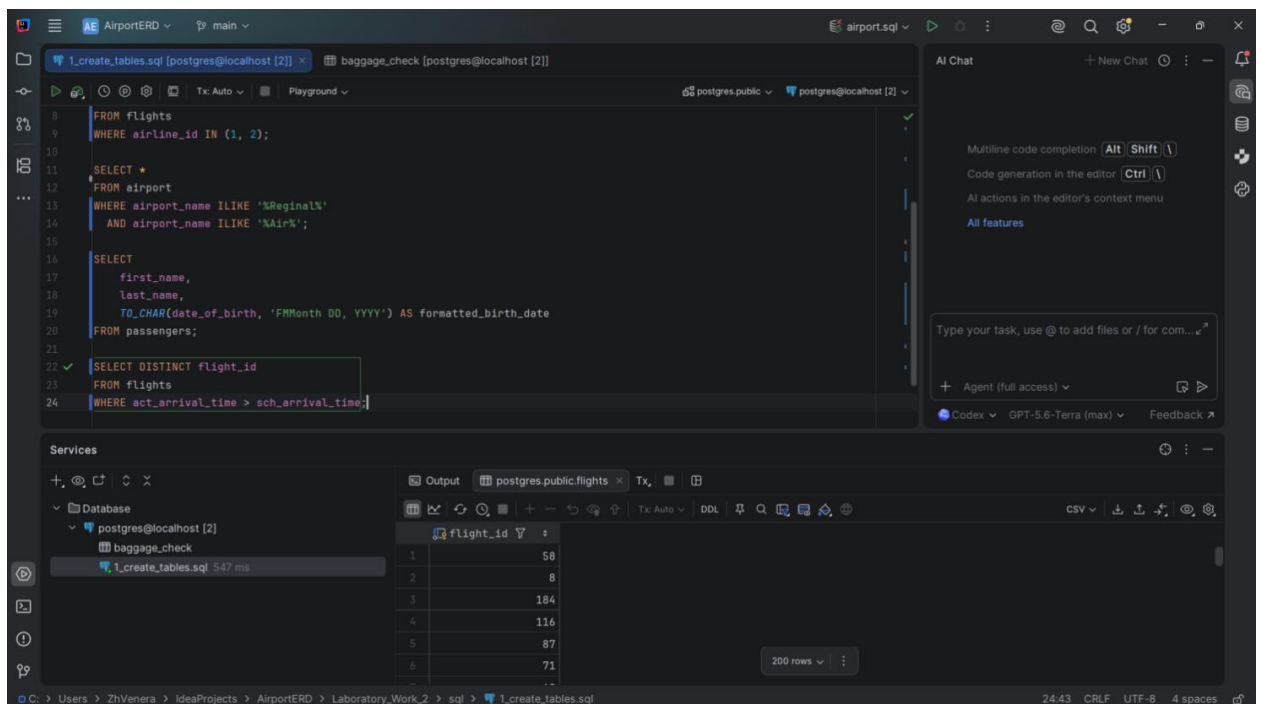


6. Find flight numbers that have been delayed based on the actual arrival time.

SELECT DISTINCT flight_id

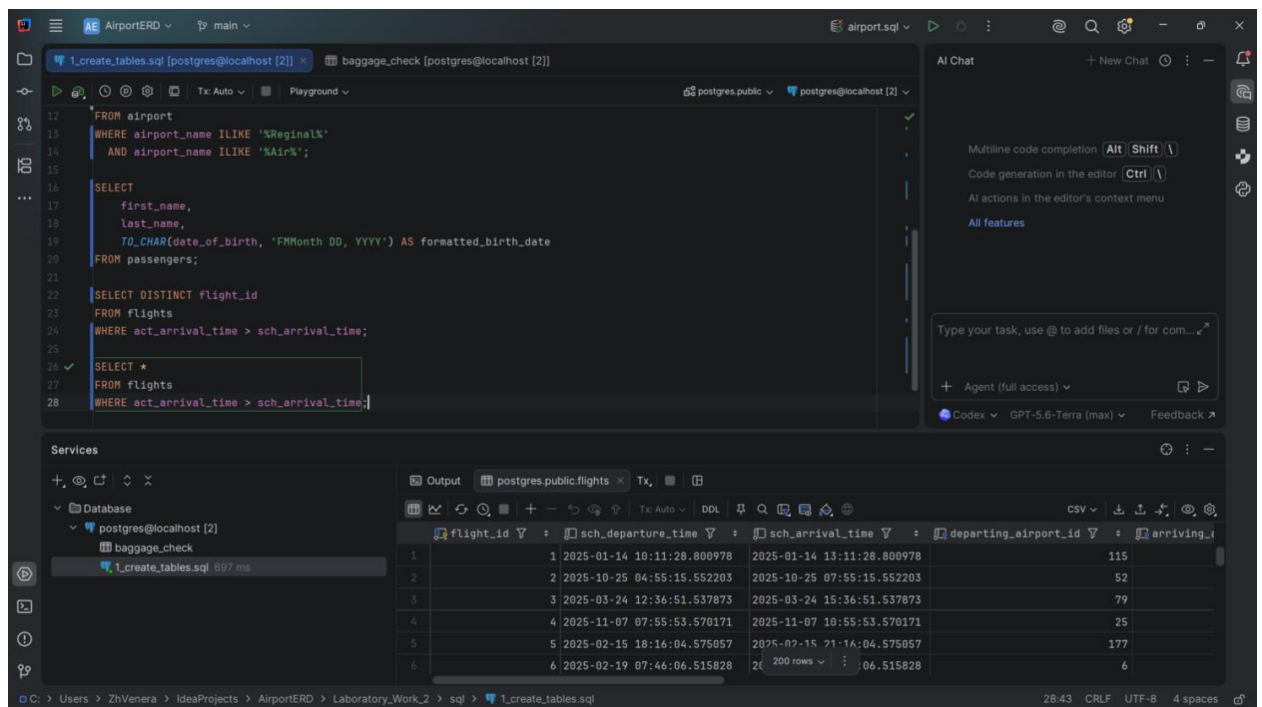
FROM flights

WHERE act_arrival_time > sch_arrival_time;



7. Find flights that arrived late according to their actual arrival time compared to the scheduled arrival time.

SELECT * FROM flights WHERE act_arrival_time > sch_arrival_time;



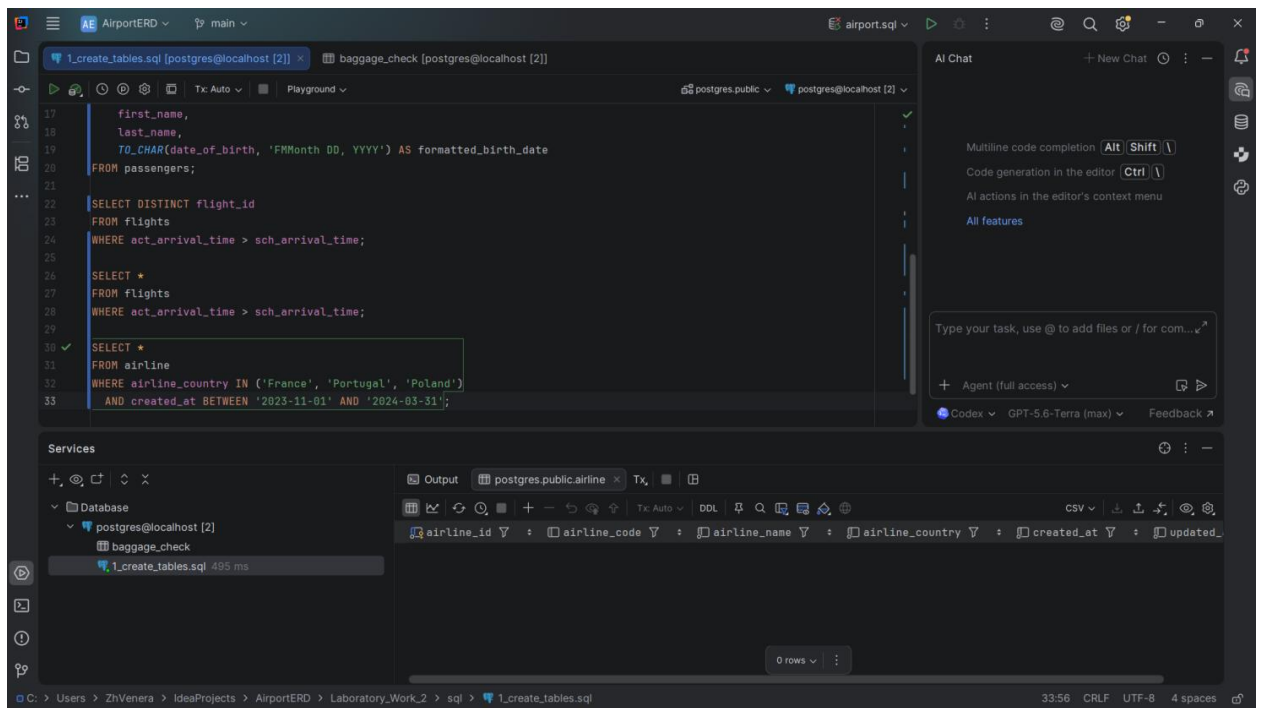
8. Show airlines from any of: ('France','Portugal','Poland') created between '2023-11-01' and '2024-03-31'.

SELECT *

FROM airline

WHERE airline_country IN ('France', 'Portugal', 'Poland')

AND created_at BETWEEN '2023-11-01' AND '2024-03-31 23:59:59';



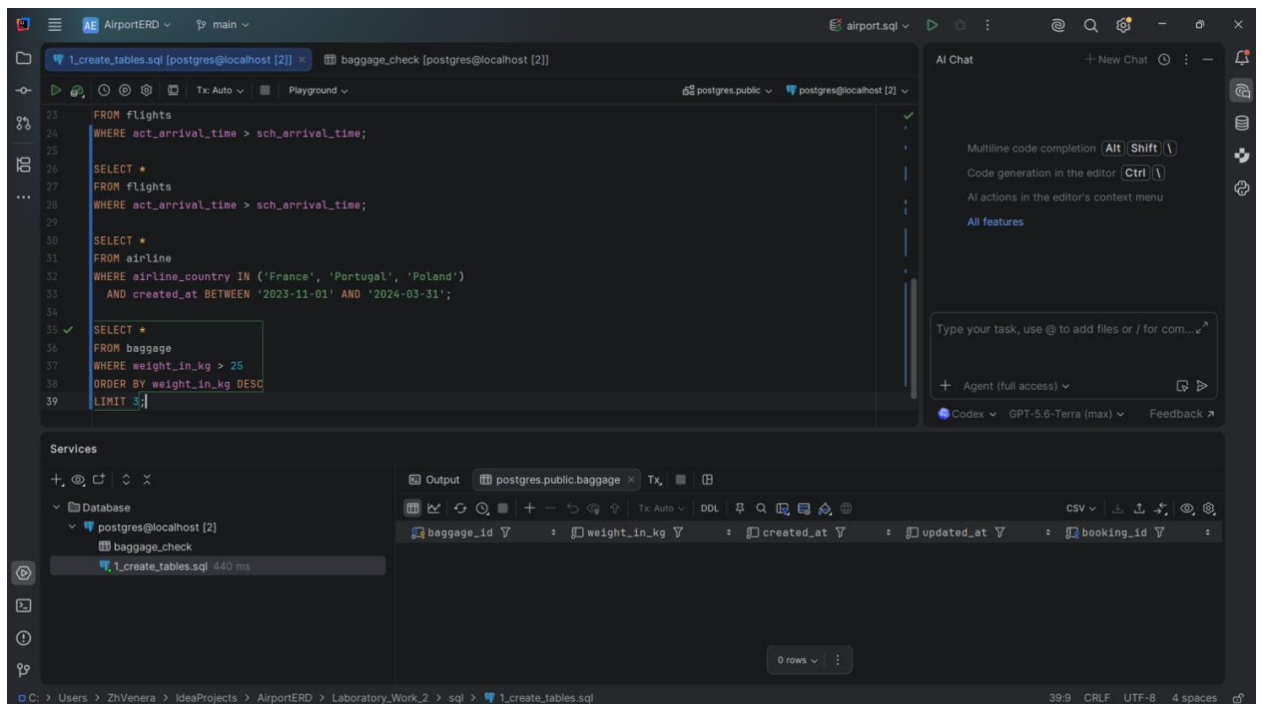
9. Find top3 overweighted baggage with more than 25kg.

SELECT *

FROM baggage

WHERE weight_in_kg > 25

ORDER BY weight_in_kg DESC LIMIT 3;



10. Find the youngest passengers' full name.

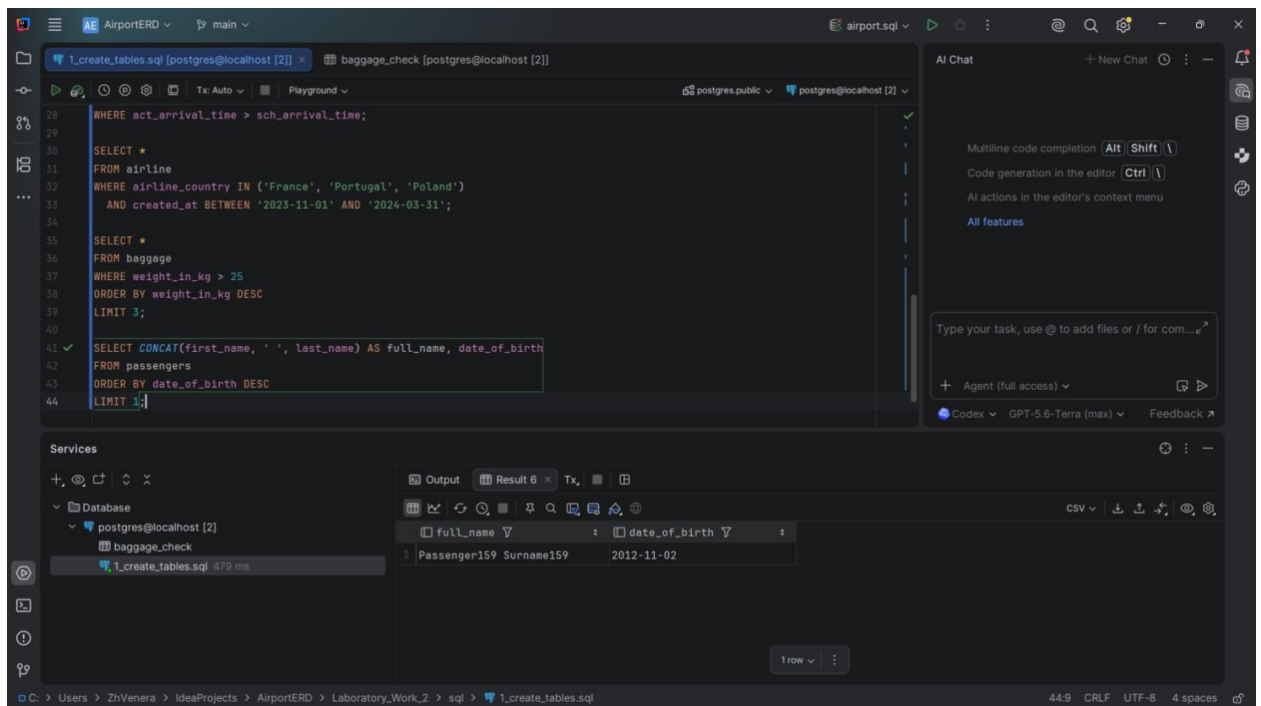
```
SELECT CONCAT(first_name, ' ', last_name)
```

```
AS full_name, date_of_birth
```

```
FROM passengers
```

```
ORDER BY date_of_birth DESC
```

```
LIMIT 1;
```



11. Find the cheapest booking price on each booking platform.

```
SELECT
```

```
booking_platform,
```

```
MIN(ticket_price) AS cheapest_price
```

```
FROM booking
```

```
GROUP BY booking_platform;
```

The screenshot shows the AirportERD IDE interface. The main editor displays three SQL queries. The first query filters baggage by weight. The second query concatenates first and last names from the passengers table. The third query, which is highlighted, selects the booking platform and the cheapest price from the booking table, grouped by platform. The 'Services' panel on the left shows the database structure. The 'Output' panel on the right displays the results of the third query as a table with two rows: 'Website' with a price of 12953.89 and 'Mobile App' with a price of 14370.6. An AI Chat panel is visible on the far right.

```
SELECT *
FROM baggage
WHERE weight_in_kg > 25
ORDER BY weight_in_kg DESC
LIMIT 3;

SELECT CONCAT(first_name, ' ', last_name) AS full_name, date_of_birth
FROM passengers
ORDER BY date_of_birth DESC
LIMIT 1;

SELECT
    booking_platform,
    MIN(ticket_price) AS cheapest_price
FROM booking
GROUP BY booking_platform;
```

booking_platform	cheapest_price
Website	12953.89
Mobile App	14370.6

12. Return airlines whose airline_code contains a digit.

SELECT *

FROM airline

WHERE airline_code ~ '[0-9]';

This screenshot shows the same AirportERD IDE interface, but the third SQL query has been updated to find airlines whose codes contain a digit. The query is: `SELECT * FROM airline WHERE airline_code ~ '[0-9]';`. The 'Output' panel now shows an empty table with 0 rows, indicating that no airlines in the current dataset have a digit in their code. The 'Services' panel and the AI Chat panel remain the same as in the previous screenshot.

```
ORDER BY weight_in_kg DESC
LIMIT 3;

SELECT CONCAT(first_name, ' ', last_name) AS full_name, date_of_birth
FROM passengers
ORDER BY date_of_birth DESC
LIMIT 1;

SELECT
    booking_platform,
    MIN(ticket_price) AS cheapest_price
FROM booking
GROUP BY booking_platform;

SELECT *
FROM airline
WHERE airline_code ~ '[0-9]';
```

airline_id	airline_code	airline_name	airline_country	created_at	updated_at
------------	--------------	--------------	-----------------	------------	------------

13. List the top5 most recently created airlines.

```
SELECT *
```

```
FROM airline
```

```
ORDER BY created_at
```

```
DESC LIMIT 5;
```

The screenshot shows an IDE with a SQL editor on the left and a database query result on the right. The SQL editor contains the following code:

```
43 ORDER BY date_of_birth DESC
44 LIMIT 1;
45
46 SELECT
47     booking_platform,
48     MIN(ticket_price) AS cheapest_price
49 FROM booking
50 GROUP BY booking_platform;
51
52 SELECT *
53 FROM airline
54 WHERE airline_code ~ '[0-9]+';
55
56 SELECT *
57 FROM airline
58 ORDER BY created_at DESC
59 LIMIT 5;
```

The database query result shows the following data:

airline_id	airline_code	airline_name	airline_country	created_at
5	GA	Global Airways	United Kingdom	2026-09-29 03:34:46.939442
2	AE	AirEasy	FRANCE	2026-09-27 13:36:29.414019
3	FH	FlyHigh	BRAZIL	2026-09-27 13:36:29.414019
4	UNK	FlyFly	POLAND	2026-09-27 13:36:29.414019
1	KAZ	KazAir	TURKEY	2026-09-27 13:34:05.365117

14. Return all rows where booking_id is between 200 and 300 inclusive and check_result <> 'Checked'.

```
SELECT * FROM baggage_check WHERE booking_id BETWEEN 200 AND
300 AND check_result <> 'Checked';
```

The screenshot displays the IntelliJ IDEA IDE interface. The main editor window shows a SQL query in the '1_create_tables.sql' file, connected to a PostgreSQL database at 'localhost [2]'. The query is as follows:

```

48 MIN(ticket_price) AS cheapest_price
49 FROM booking
50 GROUP BY booking_platform;
51
52 SELECT *
53 FROM airline
54 WHERE airline_code ~ '[0-9]';
55
56 SELECT *
57 FROM airline
58 ORDER BY created_at DESC
59 LIMIT 5;
60
61 SELECT *
62 FROM baggage_check
63 WHERE booking_id BETWEEN 200 AND 300
64 AND check_result <> 'Checked';

```

On the right side, the 'AI Chat' sidebar is open, showing a 'New Chat' button and a list of features: 'Multiline code completion' (Alt+Shift+I), 'Code generation in the editor' (Ctrl+I), 'AI actions in the editor's context menu', and 'All features'. Below this is a text input field for a task and a dropdown menu for the AI agent, currently set to 'GPT-5.6-Terra (max)'.

At the bottom, the 'Services' pane shows a tree view of the database structure. Under 'Database', there is a 'postgres@localhost [2]' connection, which contains a 'baggage_check' table. The table's columns are listed in the 'Output' pane below: 'baggage_check_id', 'check_result', 'created_at', 'updated_at', 'booking_id', and 'passenger'.

15. Baggage checks where update_at is in the same month as created_at but occurs earlier than created_at.

SELECT *

FROM baggage_check

WHERE DATE_TRUNC('month', updated_at) = DATE_TRUNC('month', created_at) AND updated_at < created_at;

